ADGSTUDIOS

DisputeGuard

Equations and Algorithms

Plain-language guide and implementation pseudocode

Companion to the DisputeGuard Software Engineering Specification

Version 1.0 — October 2026

A practical reference for the calculations, safety rules and workflows used in dispute management.

Contents

1 How to Use This Guide **2**
1.1 Notation at a Glance . . . 2

2 Money and Exact Amounts **2**

3 Ownership and Permission **2**

4 Dispute Lifecycle **3**

5 One Active Dispute per Transaction **3**

6 Safe, Repeatable Requests **3**

7 Concurrent Updates **4**

8 Deterministic Risk Indicators **4**
8.1 Amount Normalisation . . . 4
8.2 Historical Amount Anomaly . . . 4
8.3 Robust Amount Anomaly . . . 5
8.4 Transaction Velocity . . . 5
8.5 Geographical Signal . . . 5

9 Evidence Upload Workflow **6**

10 Dispute Creation Algorithm **6**

11 Authorised State Transition Algorithm **7**

12 Risk Calculation Algorithm **8**

13 Bounded AI Investigation Algorithm **8**

14 Safe Background Retries **9**

15 Key Safety Invariants **9**

1 How to Use This Guide

This is a standalone companion to the DisputeGuard engineering specification. It explains the main equations in everyday language and gives pseudocode for important system workflows.

The pseudocode describes required behaviour, not a particular programming language or database library. The main specification remains the source of project requirements. Where a business rule is not yet specified, this guide names the rule as a policy input instead of making up a threshold or decision.

1.1 Notation at a Glance

Symbol	Meaning
u	A user
t	A financial transaction
d	A dispute about a transaction
s	A dispute state
e	An event or requested action
$R(d)$	A deterministic risk indicator for dispute d
$\perp$	An invalid or undefined result

2 Money and Exact Amounts

For a currency that has q minor-unit decimal places, let $e_c = 10^q$. An amount a, parsed as an exact decimal, is represented in integer minor units as:

$$a_{\text{minor}} = a \times e_c.$$

For a two-decimal currency:

$$\text{R1234.56} \longrightarrow 123456 \text{ cents.}$$

In plain language: Store cents as whole numbers instead of binary floating-point numbers. This prevents rounding surprises when adding or comparing money. Use the decimal-place rule for the actual currency; do not assume every currency has two decimal places. Only add amounts after confirming they use the same currency.

3 Ownership and Permission

Let $Owner(u, t)$ mean that user u owns the account associated with transaction t. A customer may access their own transaction; an analyst or administrator may access it only when their role grants that permission:

$$CanView(u, t) \iff Owner(u, t) \vee Role(u) \in \{\text{ANALYST}, \text{ADMIN}\}.$$

For customer-created disputes, ownership is mandatory:

$$CanCreateDispute(u, t) \Rightarrow Owner(u, t).$$

In plain language: A transaction ID supplied by a browser is not proof that the customer owns that transaction. The server checks ownership using the authenticated user's identity on every request. Hiding a button in the user interface is not an access-control check.

4 Dispute Lifecycle

Let S be the configured set of dispute states and E the set of possible events. The transition function is:

$$T : S \times E \to S \cup \{\bot\}.$$

Some transitions specified for this system are:

$$\begin{aligned} T(\text{SUBMITTED}, \text{BEGIN_REVIEW}) &= \text{UNDER_REVIEW}, \\ T(\text{UNDER_REVIEW}, \text{REQUEST_EVIDENCE}) &= \text{EVIDENCE_REQUIRED}, \\ T(\text{EVIDENCE_REQUIRED}, \text{EVIDENCE_SUBMITTED}) &= \text{EVIDENCE_RECEIVED}, \\ T(\text{INVESTIGATING}, \text{APPROVE}) &= \text{APPROVED}, \\ T(\text{INVESTIGATING}, \text{REJECT}) &= \text{REJECTED}. \end{aligned}$$

An action not present in the authorised transition table is invalid. For example:

$$T(\text{SUBMITTED}, \text{ISSUE_REFUND}) = \bot.$$

In plain language: A case can only move through steps that the workflow allows. Every attempted change is checked by the server; the user interface cannot skip a review step. The full transition table and role permissions must be configured by the application.

5 One Active Dispute per Transaction

Let S_A be the states considered active by the application. For every transaction t, the number of active disputes is limited to one:

$$\forall t, \quad |\{d : d.transaction = t \wedge d.status \in S_A\}| \leq 1.$$

In plain language: Two requests arriving at nearly the same time must not create two open cases for the same transaction. The database must enforce this rule; an application-level check alone can lose a race.

6 Safe, Repeatable Requests

For an idempotency key K, authenticated user u, canonical request hash h, and original result r, the server records:

$$(u, K, h) \mapsto r.$$

On a retry:

$$\begin{cases} \text{return } r, & \text{if } u, K, h \text{ match a completed request,} \\ \text{reject,} & \text{if the key is reused with a different request,} \\ \text{process once,} & \text{if no matching request exists.} \end{cases}$$

In plain language: If a customer clicks submit twice or the network retries a request, the customer gets the original result rather than a duplicate dispute. Reusing the same key for different request contents must not silently return or create the wrong result.

7 Concurrent Updates

Each dispute has an integer version v. A change succeeds only when the version the caller saw still matches the database version:

$$v_{\text{request}} = v_{\text{database}}.$$

On success:

$$v_{\text{new}} = v_{\text{old}} + 1.$$

In plain language: If two analysts edit a case at once, the second save must notice that the case changed since it was opened. It should ask the user to reload or resolve the conflict, not overwrite a newer decision.

8 Deterministic Risk Indicators

Let $A, V, M, H, G \in [0, 1]$ be available, normalised signals for amount, velocity, merchant, historical behaviour and geography. Let the configured weights be non-negative and sum to one:

$$w_A + w_V + w_M + w_H + w_G = 1, \qquad w_A, w_V, w_M, w_H, w_G \geq 0.$$

The weighted score is:

$$R(d) = w_A A(d) + w_V V(d) + w_M M(d) + w_H H(d) + w_G G(d).$$

When every input is in $[0, 1]$, this implies:

$$0 \leq R(d) \leq 1.$$

In plain language: The score combines several transparent signals using configured weights. It helps an analyst decide what to look at; it does not prove fraud and must not approve or reject a dispute. A missing signal is unknown, not zero. Do not publish a combined score when required inputs are missing unless an explicit, documented missing-data policy says how to handle them.

8.1 Amount Normalisation

For a non-negative amount x and a positive configured scale $x_{\max}$:

$$A(x) = \min\left(1, \frac{\ln(1 + x)}{\ln(1 + x_{\max})}\right).$$

In plain language: This compresses very large values so one unusually large purchase cannot automatically dominate the other risk signals. The scale must be configured for the relevant currency and population.

8.2 Historical Amount Anomaly

For $n \geq 2$ prior amounts $x_1, \ldots, x_n$, the sample mean and standard deviation are:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}.$$

When $s > 0$, the standardised deviation for a new amount x is:

$$z = \frac{x - \bar{x}}{s}.$$

An absolute z-score above 3 may be labelled unusual for demonstration purposes; it is not evidence that fraud occurred. If there is insufficient history or $s = 0$, the result is unavailable rather than an invented score.

In plain language: Compare a new amount with the customer's past amounts. A value far from the usual range is a reason to take a closer look, not a verdict.

8.3 Robust Amount Anomaly

To reduce the influence of extreme past values, define the median m, the median absolute deviation MAD, and, when $MAD > 0$:

$$m = \text{median}(x_1, \ldots, x_n), \qquad MAD = \text{median}(|x_i - m|),$$

$$z_i^* = \frac{0.6745(x_i - m)}{MAD}.$$

In plain language: The median-based calculation is less affected by a few unusually large historical purchases. If $MAD = 0$, the calculation is undefined; do not divide by zero.

8.4 Transaction Velocity

For a window of length $\Delta t > 0$, let $N_u(t - \Delta t, t]$ count a customer's transactions in that window:

$$V_u(t) = \frac{N_u(t - \Delta t, t]}{\Delta t}.$$

When historical velocity has standard deviation $\sigma_{V,u} > 0$, compare current velocity with history:

$$z_V = \frac{V_u(t) - \mu_{V,u}}{\sigma_{V,u}}.$$

In plain language: Count transactions in a defined time window, then compare that rate with the customer's normal rate. The window and any threshold must be explicit configuration, not an unstated assumption.

8.5 Geographical Signal

For two latitude/longitude points in radians, let $\Delta\phi = \phi_2 - \phi_1$ and $\Delta\lambda = \lambda_2 - \lambda_1$. The Haversine calculation is:

$$\begin{aligned} a &= \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right), \\ c &= 2\,\text{atan2}\left(\sqrt{a}, \sqrt{1-a}\right), \\ d &= R_E c, \qquad R_E \approx 6371 \text{ km}, \\ v_{\text{required}} &= \frac{d}{\Delta t}, \qquad \Delta t > 0. \end{aligned}$$

In plain language: Estimate how far apart two recorded locations are, then how quickly someone would need to travel between them. Missing locations, non-positive elapsed time, online purchases, VPN use and delayed settlement make this signal uncertain. It is never proof of fraud by itself.

9 Evidence Upload Workflow

In plain language: Treat a file as untrusted until the server has checked who uploaded it, how large it is, what it contains and whether scanning is required and available. A filename or browser MIME-type claim is not proof of file content.

Listing 1: Validating and recording evidence

```
function receiveEvidence(user, disputeId, upload):
    authenticate(user)
    dispute = loadDispute(disputeId)
    require canAccessAndSubmitEvidence(user, dispute)

    require upload.size <= configuredSizeLimit
    require upload.declaredType in configuredAllowedTypes
    require extensionAllowed(upload.name)
    require detectedContentType(upload.bytes) in configuredAllowedTypes

    storageName = generateRandomServerSideName()
    storeOutsideExecutableWebPaths(storageName, upload.bytes)

    if malwareScanningIsRequired:
        require scannerIsAvailable()
        require scan(storageName) == CLEAN

    begin database transaction
        saveEvidenceMetadata(dispute, storageName, upload)
        appendEvidenceAuditEvent(user, dispute)
    commit

    return evidenceReceipt()
```

If storage or scanning succeeds but metadata persistence fails, the implementation must clean up or safely reconcile the unreferenced object. Scanner unavailability must not be reported as a clean scan.

10 Dispute Creation Algorithm

In plain language: Verify the customer's identity and ownership, apply the configured eligibility rules, and create the case and its audit records as one database operation. The database, not a pre-check in the interface, protects against simultaneous duplicate submissions.

Listing 2: Creating a dispute safely

```
function createDispute(authenticatedUser, transactionId, request, key):
    require authenticatedUser is authenticated
    canonicalRequest = canonicalize(request, transactionId)
    requestHash = hash(canonicalRequest)

    begin database transaction

    replay = findIdempotentResult(authenticatedUser.id, key)
    if replay exists:
        if replay.requestHash != requestHash:
            rollback
            reject key reuse with different request
        commit
        return replay.originalResult

    transaction = loadTransactionForUpdate(transactionId)
```

```
    require transaction exists
    require transaction.customerId == authenticatedUser.id
    require eligibilityPolicyAllows(transaction, request.reason)
    require databaseAllowsNoOtherActiveDispute(transactionId)

    dispute = insertDispute(transaction, authenticatedUser, request)
    appendDisputeEvent(dispute, "SUBMITTED", authenticatedUser)
    appendAuditEvent("DISPUTE_CREATED", authenticatedUser, dispute)
    saveIdempotentResult(authenticatedUser.id, key, requestHash, dispute
        )

    commit
    return dispute

on any failure:
    rollback all writes
    return an explicit error
```

The idempotency record and dispute must be committed atomically. The transaction lock and database uniqueness constraint are both needed: the lock serialises the competing operation, while the constraint remains the final guard against races.

11 Authorised State Transition Algorithm

In plain language: Load the latest case, confirm the actor is allowed to perform the requested action, check that the action is valid from the current state, and save the new state together with its audit record. Reject stale or invalid changes.

Listing 3: Changing a dispute state

```
function transitionDispute(actor, disputeId, event, expectedVersion):
    require actor is authenticated
    begin database transaction

    dispute = loadDisputeForUpdate(disputeId)
    require actorMayPerform(actor, event, dispute)
    require dispute.version == expectedVersion

    nextState = authorisedTransition(dispute.state, event)
    require nextState exists

    oldState = dispute.state
    update dispute:
        state = nextState
        version = version + 1
    appendDisputeEvent(dispute, oldState, nextState, actor, event)
    appendAuditEvent("DISPUTE_TRANSITION", actor, dispute)

    commit
    return updated dispute

on any failed check:
    rollback
    return an explicit error
```

12 Risk Calculation Algorithm

In plain language: Calculate each signal with its own defined inputs and safe mathematical guards. If a necessary input is missing or undefined, show that fact rather than quietly treating it as a low-risk value.

Listing 4: Calculating an explainable risk indicator

```
function calculateRisk(dispute):
    signals = {
        amount: calculateAmountSignal(dispute),
        velocity: calculateVelocitySignal(dispute),
        merchant: calculateMerchantSignal(dispute),
        history: calculateHistorySignal(dispute),
        geography: calculateGeographySignal(dispute)
    }

    if any required signal is unavailable:
        return {
            score: unavailable,
            signals: signals,
            explanation: "One or more required inputs are unavailable"
        }

    require every signal is in [0, 1]
    require configured weights are nonnegative
    require sum(configured weights) == 1

    score = sum(weight[name] * signals[name] for each signal)
    return {score: score, signals: signals}
```

Risk bands and thresholds are configuration for analyst triage. The result is not a fraud finding or an automated financial decision.

13 Bounded AI Investigation Algorithm

In plain language: An AI helper may gather a limited amount of approved information and suggest a summary. It cannot grant itself new tools, run arbitrary database queries, or make the final decision. A human with the right permissions remains responsible for case outcomes.

Listing 5: Running a bounded investigation

```
function investigate(caseId, configuredBudget):
    context = loadOnlyAuthorisedCaseContext(caseId)
    state = initialiseInvestigation(context, configuredBudget)

    while not state.complete:
        require state.iterations < configuredBudget.maxIterations
        require state.toolCalls < configuredBudget.maxToolCalls
        require state.tokens < configuredBudget.maxTokens
        require elapsedTime(state) < configuredBudget.maxDuration

        proposal = requestNextStructuredStep(state)
        require validateSchema(proposal)
        require proposal.tool in applicationAllowlist

        result = executeReadOnlyToolForThisCase(proposal)
        state = recordStructuredObservation(state, proposal, result)

    recommendation = validateSchema(requestFinalSummary(state))
```

```
    persistSummaryAndToolHistory(recommendation, state)
    return recommendation for authorised human review

on budget exhausted, invalid output, or model failure:
    record analysis as incomplete or failed
    do not change dispute state or financial balances
```

Customer descriptions and uploaded evidence are data, not instructions that can grant authority. The application enforces tool permissions and execution bounds independently of model prompts.

14 Safe Background Retries

In plain language: A worker can receive the same job more than once after a timeout or crash. Before retrying, the application must know that repeating the operation cannot duplicate an external effect.

Listing 6: Handling retryable background work

```
function handleJob(job):
    require job.operation is explicitly retryable
    require job has a stable idempotency identifier

    if durableResultExists(job.idempotencyIdentifier):
        acknowledge job
        return stored result

    try:
        result = performOperationWithIdempotency(job)
        saveDurableResult(job.idempotencyIdentifier, result)
        acknowledge job
    catch retryableFailure:
        retry only while configured retry policy permits
        otherwise record failure for operator review
    catch nonRetryableFailure:
        record failure for operator review
        do not retry automatically
```

Exactly-once effects across independent services cannot be assumed from a queue alone. Durable idempotency records, transactional writes and explicit failure handling are needed.

15 Key Safety Invariants

Invariant	What it means in practice
Money is exact	Use integer minor units or fixed-precision decimal values.
Ownership is server-checked	A caller cannot gain access by changing an identifier.
At most one active dispute	Database constraints protect simultaneous submissions.
Transitions are authorised	Only configured actor/event/state combinations can proceed.
Retries are safe	A repeated request returns its original result, not a duplicate effect.
Risk is advisory	A score or anomaly is not proof and cannot decide a case.
AI is bounded	Allowlisted tools and resource limits are enforced by application code.
Audit is durable	Case changes and their audit records are saved together.